

# Basic Operators, Maths - Simple Hinglish Summary

Source: <https://javascript.info/operators>

## Main baat

School wale operators (+, -, \*) hum jaante hain. Is chapter me simple operators ke saath JavaScript ki khaas baatein seekhenge.

### 1. Terms: operand, unary, binary

- **Operand:** jis par operator lagta hai.  $5 * 2$  me 5 aur 2 operands hain.
- **Unary:** jis operator ka ek operand ho. Jaise  $-x$  (sign ulta karta hai).
- **Binary:** jis operator ke do operands hon. Jaise  $y - x$ .

```
javascript

let x = 1;
x = -x;           // -1 (unary minus)

let y = 3;
alert( y - x );   // binary minus
```

### 2. Maths operators

+ (add), - (subtract), \* (multiply), / (divide), % (remainder), \*\* (power)

- **Remainder %:** percent se koi lena-dena nahi.  $a \% b$  = a ko b se divide karne par bacha hua hissa.

```
javascript

alert( 5 % 2 ); // 1
alert( 8 % 3 ); // 2
alert( 8 % 4 ); // 0
```

- **Exponentiation \*\*:**  $a ** b$  = a ki power b.

```
javascript

alert( 2 ** 3 ); // 8
alert( 4 ** (1/2) ); // 2 (square root)
alert( 8 ** (1/3) ); // 2 (cube root)
```

### 3. String concatenation (binary $\oplus$ )

Strings par  $\oplus$  lagao to wo **jod (merge)** deta hai:

```
javascript  
  
let s = "my" + "string"; // mystring
```

Agar **koi ek bhi operand string** ho, to dusra bhi string me convert ho jata hai:

```
javascript  
  
alert( '1' + 2 ); // "12"  
alert( 2 + '1' ); // "21"
```

Operators **ek ke baad ek (left to right)** chalte hain:

```
javascript  
  
alert( 2 + 2 + '1' ); // "41" (pehle 2+2=4, phir 4+'1')  
alert( '1' + 2 + 2 ); // "122" (pehle '1'+2='12', phir '12'+2)
```

**Sirf binary  $\oplus$**  aisa karta hai. Baaki operators ( $\ominus$ ,  $\oslash$ , etc.) hamesha **numbers me convert** karte hain:

```
javascript  
  
alert( 6 - '2' ); // 4  
alert( '6' / '2' ); // 3
```

### 4. Unary $\oplus$ (numeric conversion)

Number par kuch nahi karta, lekin **non-number ko number bana deta hai** (`Number(...)` jaisa, par chhota):

```
javascript  
  
alert( +true ); // 1  
alert( +"" ); // 0
```

**Kaam kab aata hai:** form se values string me aati hain. Direct  $\oplus$  karoge to jud jayengi:

```
javascript
```

```
let apples = "2";
let oranges = "3";

alert( apples + oranges );    // "23" (galat)
alert( +apples + +oranges ); // 5    (sahi)
```

## 5. Operator precedence (kaun pehle chalega)

- Jaise  $1 + 2 * 2$  me  $*$  pehle chalta hai (**higher precedence**).
- Brackets**  $( )$  kisi bhi precedence ko override kar dete hain:  $(1 + 2) * 2$
- Same precedence ho to **left se right** chalta hai.

| Precedence | Operator              |
|------------|-----------------------|
| 14         | unary $+$ , unary $-$ |
| 13         | $**$                  |
| 12         | $*$ , $/$             |
| 11         | binary $+$ , $-$      |
| 2          | $=$ (assignment)      |

Isliye  $+apples + +oranges$  me unary  $+$  (14) pehle chalta hai, binary  $+$  (11) baad me.

## 6. Assignment $=$ bhi operator hai

- Precedence bahut kam (2), isliye pehle calculation hoti hai, phir value store hoti hai.

javascript

```
let x = 2 * 2 + 1; // 5
```

- Assignment value return karta hai.**  $x = \text{value}$  value ko x me likhta hai aur **phir return bhi karta hai**.

javascript

```
let a = 1;
let b = 2;
let c = 3 - (a = b + 1);

alert( a ); // 3
alert( c ); // 0
```

Ye code libraries me dikhta hai, lekin **khud aisa mat likho** (readable nahi).

## Chained assignment

**Right se left** chalta hai:

javascript

```
let a, b, c;
a = b = c = 2 + 2; // teeno 4
```

Readability ke liye alag lines me likhna behtar hai.

## 7. Modify-in-place (`+=`, `*=`, etc.)

javascript

```
let n = 2;
n += 5; // n = n + 5 → 7
n *= 2; // n = n * 2 → 14
```

Sabhi arithmetic operators ke liye hain: `-=`, `/=`, etc. Precedence assignment jaisi hoti hai (right side pehle calculate hoti hai):

javascript

```
let n = 2;
n *= 3 + 5; // n *= 8 → 16
```

## 8. Increment / Decrement (`++`, `--`)

- `++` variable ko **1 badhata** hai, `--` **1 ghatata** hai.
- Sirf variable par** lagta hai (`5++` error hai).

javascript

```
let counter = 2;
counter++;    // 3
counter--;    // 2
```

## Prefix vs Postfix

- **Postfix:** `counter++` (variable ke baad)
- **Prefix:** `++counter` (variable se pehle)

Dono variable ko badhate hain. Fark sirf **return value** me hai:

- **Prefix** → **nayi** value return karta hai
- **Postfix** → **purani** value return karta hai

javascript

```
let counter = 1;
let a = ++counter; // a = 2 (nayi value)

let counter2 = 1;
let b = counter2++; // b = 1 (purani value)
```

## Yaad rakhne ka tarika:

- Result use nahi kar rahe → dono same.
- Badha kar **turant nayi value** chahiye → **prefix** (`++counter`)
- **Purani value** chahiye, phir badhana hai → **postfix** (`counter++`)

**Achhi aadat:** "one line - one action". `2 * counter++` jaise code readable nahi hote.

## 9. Bitwise operators

`&`, `|`, `^`, `~`, `<<`, `>>`, `>>>`. Ye numbers ko **32-bit integers** ki tarah binary level par treat karte hain. Web development me **bahut kam** kaam aate hain (cryptography jaise special areas me useful).

## 10. Comma operator `,`

Bahut rare operator. Kai expressions chalata hai, lekin **sirf aakhri ka result return** karta hai.

javascript

```
let a = (1 + 2, 3 + 4);
alert( a ); // 7
```

Precedence bahut kam hai, isliye **brackets zaruri** hain. Ise samajhna zaruri hai kyunki frameworks me dikhta hai, lekin use karne se bachna chahiye.

## Practice Tasks (Answers)

### 1. Postfix aur prefix:

```
javascript

let a = 1, b = 1;
let c = ++a; // c = 2
let d = b++; // d = 1
// final: a = 2, b = 2, c = 2, d = 1
```

### 2. Assignment result:

```
javascript

let a = 2;
let x = 1 + (a *= 2);
// a = 4, x = 5
```

### 3. Type conversions ke results:

```
javascript

"" + 1 + 0      // "10"
"" - 1 + 0      // -1
true + false    // 1
6 / "3"         // 2
"2" * "3"       // 6
4 + 5 + "px"    // "9px"
"$" + 4 + 5     // "$45"
"4" - 2         // 2
"4px" - 2       // NaN
" -9 " + 5      // " -9 5"
" -9 " - 5      // -14
null + 1        // 1
undefined + 1   // NaN
" \t \n" - 2    // -2
```

### 4. Addition fix karo (prompt string deta hai):

```
javascript
```

```
let a = +prompt("First number?", 1);
let b = +prompt("Second number?", 2);

alert(a + b); // 3
```

## Quick Summary

| Topic                | Yaad rakhne wali baat                                |
|----------------------|------------------------------------------------------|
| Binary (+)           | String ho to jodta hai, warna add karta hai          |
| Baaki math operators | Hamesha number me convert karte hain                 |
| Unary (+)            | String ko number banata hai                          |
| Precedence           | Unary > (**) > (* /) > (+ -) > (=)                   |
| (=)                  | Value return karta hai, right se left chain hota hai |
| (++x) vs (x++)       | Nayi value vs purani value return                    |